UNIVERSIDAD DE GUANAJUATO
Deep Learning

Práctica 2: Forecasting de IMU

Predicción temporal con RNN y LSTM y despliegue con Triton

Forecasting de seis señales inerciales del dataset EuRoC MAV, integración cinemática de trayectoria y observabilidad MLOps.

Autor
Juan Ulises Calderón Huerta

Institución
Universidad de Guanajuato

Reporte técnico de la Práctica 2
de Deep Learning

Fecha
8 de noviembre de 2026

Índice

1. Resumen **3**

2. Introducción y objetivos **3**

3. Datos y preparación **3**

3.1. Dataset EuRoC MAV 3

3.2. Pipeline de ventanas 4

4. Modelos y entrenamiento **4**

4.1. Arquitecturas recurrentes 4

4.2. Entrenamiento y exportación 4

5. Evaluación de forecasting **5**

6. Integración cinemática y trayectoria **6**

6.1. Integración Strapdown 6

6.2. Trayectorias obtenidas 6

7. Despliegue MLOps **7**

7.1. Repositorio de modelos y Triton 7

7.2. Orquestación y observabilidad 7

8. Reproducibilidad **7**

9. Limitaciones y trabajo futuro **8**

10. Conclusiones **8**

Índice de figuras

1. Flujo común de las arquitecturas RNN y LSTM. 4
2. Comparación de las seis señales IMU en el conjunto de prueba. 5
3. Vista superior de la trayectoria integrada. Negro: Ground Truth; azul: IMU real durante Train/Val; rojo punteado: IMU predicha durante Test. 6
4. Arquitectura de inferencia y observabilidad del despliegue. 7

1. Resumen

Este reporte presenta un sistema de aprendizaje profundo para forecasting un paso adelante de señales inerciales. A partir de una ventana histórica de 50 muestras, modelos recurrentes RNN y LSTM predicen la siguiente muestra de seis variables del sensor IMU: tres velocidades angulares del giroscopio y tres aceleraciones del acelerómetro. Los datos provienen del dataset EuRoC MAV, que también proporciona la trayectoria de referencia del vehículo.

El flujo implementado cubre el ciclo completo de machine learning: carga y normalización de datos, ventanas temporales, entrenamiento en PyTorch, evaluación, exportación a ONNX y servicio con Triton Inference Server. Además, se realiza una integración cinemática tipo *strapdown* para visualizar la deriva acumulada al usar IMU real y predicha. La infraestructura de despliegue usa Docker Compose, Prometheus y Grafana.

2. Introducción y objetivos

La estimación de movimiento con una unidad de medición inercial permite obtener información de navegación con alta frecuencia, pero la integración de aceleraciones y velocidades angulares acumula error con el tiempo. En este proyecto se estudia una alternativa complementaria: aprender la dinámica local de las señales IMU para predecir la muestra siguiente y observar cómo la predicción afecta la reconstrucción de trayectoria.

Los objetivos son:

- Construir un pipeline reproducible de forecasting temporal en PyTorch.
- Comparar arquitecturas RNN y LSTM para predecir las seis señales IMU.
- Separar cronológicamente los datos en entrenamiento, validación y prueba.
- Exportar los modelos entrenados a ONNX para inferencia en Triton.
- Integrar aceleración y velocidad angular para visualizar una trayectoria 2D y su deriva respecto al Ground Truth.
- Incorporar observabilidad mediante métricas de Triton, Prometheus y Grafana.

3. Datos y preparación

3.1. Dataset EuRoC MAV

Se utilizan secuencias del dataset EuRoC MAV, organizadas en carpetas como `V1\_01\_easy`, `V1\_02\_medium` y `V1\_03\_difficult`. Cada archivo IMU contiene un timestamp en nanosegundos y las columnas:

$$\omega_x, \omega_y, \omega_z, a_x, a_y, a_z.$$

Las velocidades angulares están expresadas en radianes por segundo y las aceleraciones en metros por segundo al cuadrado. El Ground Truth paralelo `state\_groundtruth\_estimate0/data.csv` contiene posición, orientación como cuaternión y velocidad en el marco global.

3.2. Pipeline de ventanas

El módulo `src/data\_pipeline.py` divide una secuencia cronológicamente con proporciones 70 % para entrenamiento, 15 % para validación y 15 % para prueba. El escalador se calcula exclusivamente con el subconjunto de entrenamiento, evitando filtrar información del futuro:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Posteriormente se crean ventanas deslizantes de longitud $L = 50$:

$$X_t = [\tilde{x}_{t-50}, \ldots, \tilde{x}_{t-1}], \qquad y_t = \tilde{x}_t.$$

Por tanto, la entrada tiene forma $[B, 50, 6]$ y la salida forma $[B, 6]$. Los parámetros μ y σ se persisten en `data/processed/scaler.json`.

Elemento	**Configuración**
Señales de entrada	3 gyro + 3 acelerómetro
Frecuencia aproximada	200 Hz
Ventana histórica	50 muestras
Horizonte	1 muestra $(t + 1)$
Split temporal	70 % / 15 % / 15 %
Normalización	Z-score calculado en entrenamiento

Tabla 1: Configuración del problema de forecasting.

4. Modelos y entrenamiento

4.1. Arquitecturas recurrentes

La arquitectura RNN usa una capa recurrente tanh de dos capas y una capa lineal que proyecta el estado oculto final a seis salidas. La LSTM reemplaza la celda RNN por una memoria con compuertas, manteniendo la misma interfaz de entrada y salida. En ambos casos se usa `batch_first=True`.

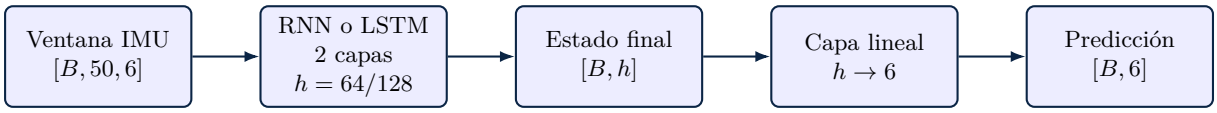


Figura 1: Flujo común de las arquitecturas RNN y LSTM.

Los checkpoints presentes muestran que el modelo LSTM fue entrenado con `hidden_dim=128` y dos capas, mientras que el RNN usa `hidden_dim=64` y dos capas. El script de carga detecta estas dimensiones desde el checkpoint para evitar incompatibilidades.

4.2. Entrenamiento y exportación

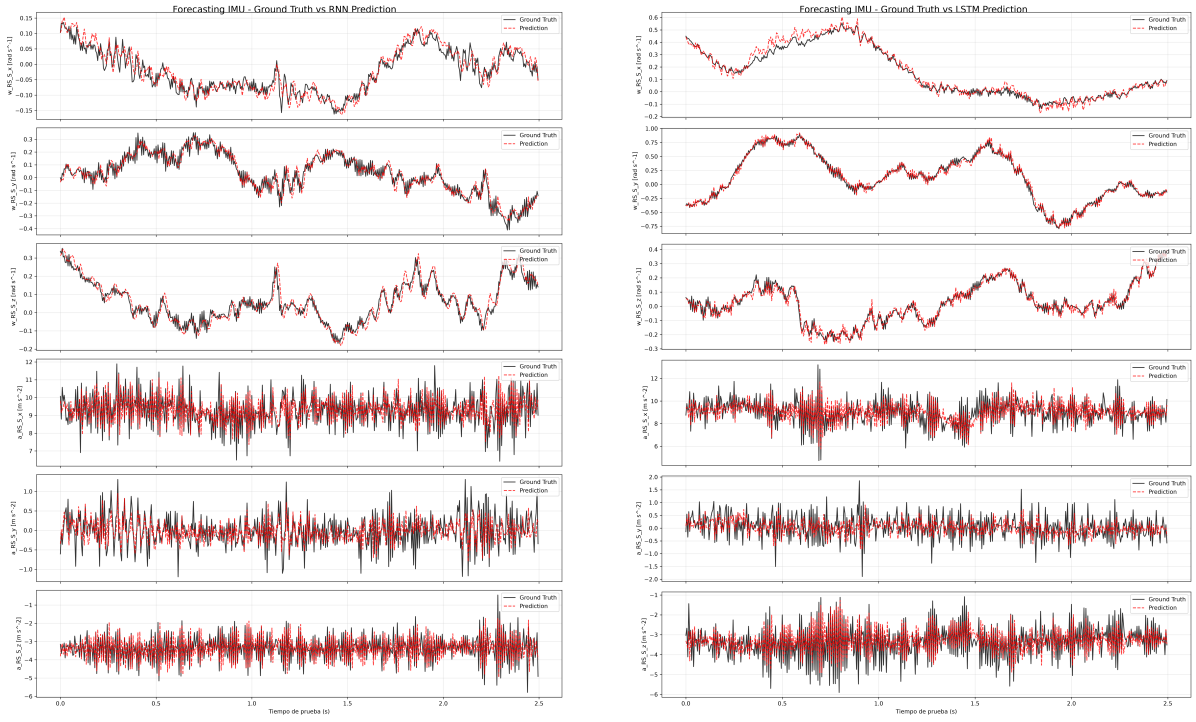
El entrenamiento se implementa en `src/train.py`. Se utiliza MSE como función de pérdida y Adam como optimizador. El mejor estado se selecciona por pérdida de validación y se guarda como `checkpoints/best\_lstm.pth` o `checkpoints/best\_rnn.pth`. Al final, el modelo se exporta a ONNX con eje de batch dinámico para Triton:

Código 1: Contrato de exportación a ONNX.

```
input_names = ["input"]
output_names = ["output"]
dynamic_axes = {
    "input": {0: "batch_size"},
    "output": {0: "batch_size"},
}
```

5. Evaluación de forecasting

El módulo `src/evaluate.py` carga el checkpoint, ejecuta inferencia sobre el segmento de prueba, revierte la normalización y calcula MSE y MAE globales y por señal. También genera una gráfica con las seis señales, comparando el valor real con la predicción del modelo.



(a) Predicción del modelo RNN. **(b)** Predicción del modelo LSTM.

Figura 2: Comparación de las seis señales IMU en el conjunto de prueba.

Las gráficas muestran que el problema es multivariable: el modelo debe predecir simultáneamente tasas angulares y aceleraciones con escalas físicas distintas. La normalización permite que todas las componentes contribuyan a la pérdida sin que una magnitud domine exclusivamente el entrenamiento. La evaluación de un paso no debe confundirse con una garantía de estabilidad al realizar predicción autoregresiva durante muchos pasos.

6. Integración cinemática y trayectoria

6.1. Integración Strapdown

El script `src/trajectory\_integration.py` construye una señal híbrida: usa IMU real durante entrenamiento y validación, y predicciones autoregresivas durante prueba. Cada predicción se obtiene con una ventana que incluye las muestras anteriores y, en Test, también las predicciones ya generadas.

Las condiciones iniciales se extraen de la fila de Ground Truth temporalmente más cercana al primer timestamp de IMU: posición $\mathbf{p}_0$, velocidad $\mathbf{v}_0$ y cuaternión $\mathbf{q}_0$. En cada paso se actualiza la orientación con la velocidad angular:

$$\mathbf{R}_k = \mathbf{R}_{k-1}\,\mathrm{Exp}\left(\boldsymbol{\omega}_k \Delta t\right), \qquad \Delta t = \frac{t_k - t_{k-1}}{10^9}.$$

La aceleración se transforma del marco del sensor al global y se compensa la gravedad:

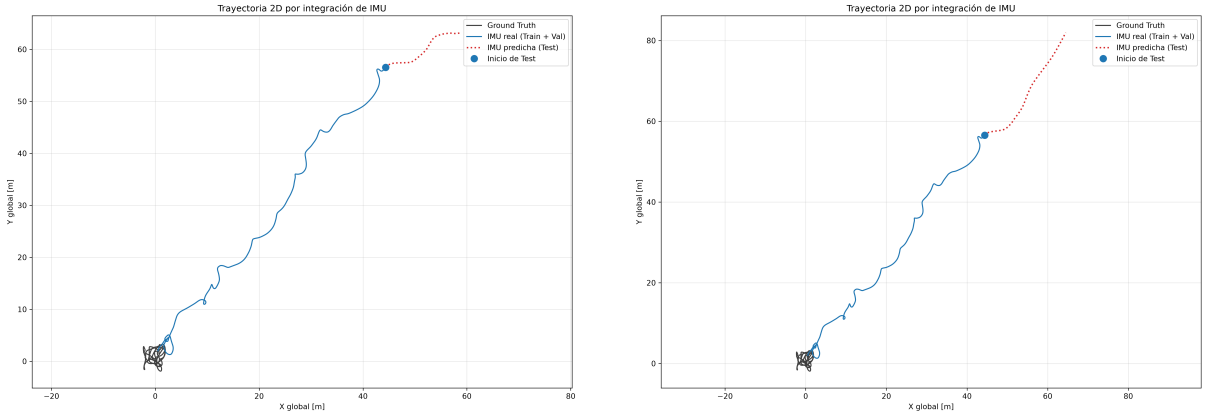
$$\mathbf{a}_k^g = \mathbf{R}_k \mathbf{a}_k^s, \qquad \mathbf{a}_k^{\mathrm{lin}} = \mathbf{a}_k^g - [0, 0, 9.81].$$

Finalmente se utiliza una actualización Euler semi-implícita:

$$\mathbf{v}_k = \mathbf{v}_{k-1} + \mathbf{a}_k^{\mathrm{lin}} \Delta t, \qquad \mathbf{p}_k = \mathbf{p}_{k-1} + \mathbf{v}_k \Delta t.$$

Esta integración es deliberadamente básica y no incluye calibración de bias, fusión con sensores externos ni corrección de deriva.

6.2. Trayectorias obtenidas



(a) Trayectoria con RNN. **(b)** Trayectoria con LSTM.

Figura 3: Vista superior de la trayectoria integrada. Negro: Ground Truth; azul: IMU real durante Train/Val; rojo punteado: IMU predicha durante Test.

La deriva observada en las curvas estimadas es esperable: un pequeño error de aceleración produce un error creciente en velocidad y un error aún mayor en posición. La visualización no pretende reemplazar un sistema de navegación inercial calibrado; su propósito es mostrar el efecto acumulativo del error y la diferencia entre integrar datos reales y datos predichos.

7. Despliegue MLOps

7.1. Repositorio de modelos y Triton

Los modelos ONNX se organizan con la estructura requerida por Triton:

Código 2: Repositorio de modelos Triton.

```
model_repository/
  lstm_forecaster/
    config.pbtxt
    1/model.onnx
  rnn_forecaster/
    config.pbtxt
    1/model.onnx
```

Ambos modelos reciben una entrada `FP32` de dimensiones $[50, 6]$ por elemento del batch y devuelven seis valores `FP32`. La configuración permite batches de hasta 256 solicitudes y activa *dynamic batching*. El cliente HTTP en `src/triton\_client.py` toma un ejemplo normalizado, construye una entrada $[1, 50, 6]$ y valida una salida $[1, 6]$.

7.2. Orquestación y observabilidad

El archivo `deploy/docker-compose.yml` levanta tres servicios:

- Triton en los puertos HTTP 8000, gRPC 8001 y métricas 8002.
- Prometheus en el puerto 9090, haciendo *scrape* de Triton.
- Grafana en el puerto 3000 con datasource y dashboard provisionados.

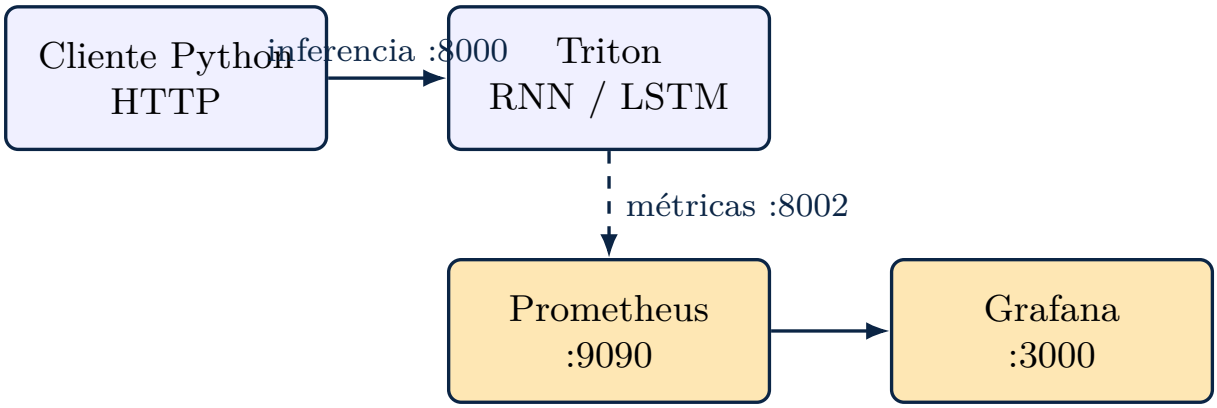


Figura 4: Arquitectura de inferencia y observabilidad del despliegue.

La imagen de Triton requiere que el host tenga NVIDIA Container Toolkit configurado cuando se utiliza GPU. La infraestructura está separada del entorno de entrenamiento para facilitar reproducibilidad y operación.

8. Reproducibilidad

Los pasos principales para reproducir la práctica desde la raíz del repositorio son:

Código 3: Comandos principales del proyecto.

```
conda activate deeplearning
pip install -r requirements.txt
python -m pytest tests/

python src/train.py --model_type lstm \
  --sequence data/raw/V1_01_easy/imu0/data.csv \
  --hidden_dim 128 --num_layers 2

python src/evaluate.py --model_type lstm \
  --sequence data/raw/V1_01_easy/imu0/data.csv

python src/trajectory_integration.py --model_type lstm \
  --sequence data/raw/V1_01_easy/imu0/data.csv

docker compose -f deploy/docker-compose.yml up -d
python src/triton_client.py --model lstm_forecaster
```

9. Limitaciones y trabajo futuro

- La integración básica no estima bias del acelerómetro ni del giroscopio; por ello la deriva puede crecer rápidamente.
- La orientación se inicializa con Ground Truth, una condición que no estaría disponible en un despliegue completamente autónomo.
- La evaluación se realiza sobre una secuencia y un split temporal; conviene reportar varias secuencias, semillas y métricas agregadas.
- El forecasting autoregresivo puede acumular errores aunque la métrica de un paso sea razonable; se requiere evaluar horizontes múltiples.
- La integración podría mejorarse con preintegración IMU, filtros de Kalman, optimización de trayectoria o fusión visual-inercial.
- El despliegue MLOps es una demostración local y no incluye pruebas de carga, alta disponibilidad ni autenticación.

10. Conclusiones

Se construyó un flujo completo para predecir señales IMU con redes recurrentes y llevar los modelos hasta inferencia servida. El proyecto conecta el procesamiento temporal de EuRoC MAV con checkpoints PyTorch, exportación ONNX, Triton, Prometheus y Grafana. La reconstrucción de trayectoria permite observar de forma concreta que los errores pequeños de las señales inerciales se acumulan al integrar.

Como resultado de la Práctica 2, el repositorio no solo contiene un modelo entrenado: también define un contrato de inferencia, un cliente de validación y una infraestructura reproducible para monitorear el servicio. Las figuras de evaluación y trayectoria incluidas en este informe documentan el comportamiento de los modelos RNN y LSTM en las condiciones experimentales descritas.